

ANGULAR

Interview Cheatsheet — Clean 2–3 Line Definitions

Core Concepts · RxJS · Component APIs · DI · Signals · Performance

What's inside

1. Core Angular Fundamentals
2. Component Communication & Decorators
3. Dependency Injection
4. Change Detection & Performance
5. RxJS & Reactive Patterns
6. Signals (Modern Angular)
7. Forms
8. Routing & Lifecycle
9. NgRx & State Management
10. Testing & Build Tooling

1. Core Angular Fundamentals

Component

A TypeScript class decorated with `@Component` that controls a portion of the UI via a template, styles, and logic. It's the basic building block of an Angular application.

Module (NgModule)

A container that groups related components, directives, pipes, and services together. Modern Angular favors standalone components over NgModules for simpler bootstrapping.

Standalone Component

A component that declares its own dependencies via the `imports` array instead of relying on an NgModule. Reduces boilerplate and is now the default in new Angular projects.

Directive

A class that adds behavior to DOM elements. Structural directives (`*ngIf`, `*ngFor`) change DOM layout; attribute directives (`ngClass`) change appearance or behavior.

Pipe

A simple function used in templates to transform displayed data, e.g. `{{ date | date }}`. Pipes can be pure (default, cached) or impure (re-run every change detection cycle).

Template Reference Variable

A variable declared with `#name` in a template that gives direct access to a DOM element or component instance within that template.

Interpolation

The `{{ expression }}` syntax used to bind component data into the template as text. Angular evaluates the expression and inserts the result into the DOM.

Property Binding

One-way binding from component to DOM using `[property]="value"`, used to set element properties, attributes, or component inputs dynamically.

Event Binding

Binds a DOM event to a component method using `(event)="handler()"`, allowing the component to react to user interactions like clicks or input.

Two-Way Binding

Combines property and event binding using `[(ngModel)]="value"`, so the UI updates the model and the model updates the UI automatically.

ng-content (Content Projection)

Lets a parent component inject markup into a placeholder inside a child component's template, enabling reusable "wrapper" style components like cards or modals.

ng-template / ng-container

`ng-template` defines a template block that isn't rendered until explicitly used; `ng-container` is a non-rendering grouping element used to apply structural directives without adding extra DOM.

2. Component Communication & Decorators

@Input()

Decorator that marks a class property as bindable from a parent component's template, allowing data to flow from parent to child.

@Output()

Decorator paired with an `EventEmitter` that lets a child component emit custom events upward to notify the parent of changes.

@ViewChild

Gets a reference to a single child component, directive, or DOM element from within the component's own template. Only available after `ngAfterViewInit`.

@ViewChildren

Similar to `@ViewChild` but returns a `QueryList` of all matching elements or components in the view, updating reactively if the DOM changes.

@ContentChild

Gets a reference to a single element or component that was projected into the component via `ng-content`, available after `ngAfterContentInit`.

@ContentChildren

Returns a `QueryList` of all projected elements/components matching a selector, useful for building tab groups, accordions, or lists of projected items.

@HostListener

Decorator that binds a method to a DOM event on the host element (or document/window), e.g. listening for clicks, resizes, or key presses without manual `addEventListener`.

@HostBinding

Decorator that binds a component/directive property to a host element's attribute, class, or style, keeping the host's DOM in sync with internal state.

ElementRef

A wrapper around a native DOM element injected into a component or directive, giving direct DOM access — used sparingly since it bypasses Angular's abstraction.

Renderer2

A safe abstraction for manipulating the DOM (instead of using `ElementRef` directly), ensuring compatibility across platforms like server-side rendering.

TemplateRef & ViewContainerRef

`TemplateRef` represents an embedded template; `ViewContainerRef` is a container where views can be dynamically created and inserted, foundational to structural directives.

3. Dependency Injection

Dependency Injection (DI)

A design pattern where a class receives its dependencies (services) from an external injector rather than creating them itself, improving testability and decoupling.

@Injectable()

Marks a class as available to be injected as a dependency and lets Angular know how to construct it, typically paired with `providedIn: 'root'` for a singleton.

providedIn: 'root'

Registers a service at the application's root injector, making it a singleton available app-wide and enabling tree-shaking if unused.

Injector Hierarchy

Angular resolves dependencies through a tree of injectors (root, module, component). A component-level provider creates a new instance scoped to that component and its children.

InjectionToken

Used to create a unique token for injecting values that aren't classes, such as configuration objects or primitives, avoiding naming collisions.

inject() function

A modern functional alternative to constructor injection that retrieves a dependency inside injection contexts (e.g. field initializers, standalone functions), commonly used with Signals.

@Optional / @Self / @SkipSelf / @Host

DI decorators that fine-tune resolution: `@Optional` allows a null dependency, `@Self / @Host` restrict lookup scope, and `@SkipSelf` skips the current injector.

Multi-Providers

Providers registered with `multi: true` so multiple values (e.g. multiple `HTTP_INTERCEPTORS`) can be injected together as an array under one token.

4. Change Detection & Performance

Change Detection

Angular's mechanism for syncing component state with the DOM. It walks the component tree checking bindings for changes whenever an event, timer, or async operation fires.

Zone.js

A library that patches async browser APIs (setTimeout, promises, events) so Angular knows when to trigger change detection automatically, without manual calls.

ChangeDetectionStrategy.OnPush

An optimization that tells Angular to check a component only when its `@Input` references change, an event originates from it, or an observable it's subscribed to (via async pipe) emits.

ChangeDetectorRef

A service that lets a component manually control change detection, using `detectChanges()` to force a check or `markForCheck()` to flag it for the next cycle under OnPush.

Zoneless Change Detection

A newer Angular mode that removes Zone.js entirely, relying on Signals and explicit notifications to know when to re-render, resulting in smaller bundles and better performance.

trackBy

A function passed to `*ngFor / @for` that tells Angular how to identify items uniquely, preventing unnecessary DOM re-creation when list data changes.

Lazy Loading

Loading feature modules or routes only when needed (via dynamic `import()`) instead of at app startup, reducing the initial bundle size and improving load time.

Pure Pipes

Pipes marked `pure: true` (the default) that only re-evaluate when their input reference changes, avoiding expensive recalculation on every change detection cycle.

5. RxJS & Reactive Patterns

Observable

A stream of values delivered over time that consumers subscribe to. It's lazy (doesn't run until subscribed) and can emit multiple values, an error, or a completion signal.

Observer / Subscription

An Observer is the set of callbacks (`next`, `error`, `complete`) passed to `subscribe()`; the returned Subscription lets you `unsubscribe()` to stop listening and prevent leaks.

Subject

Both an Observable and an Observer — it can be subscribed to and also manually pushed values via `.next()`. Unlike a plain Observable, it multicasts to all subscribers.

BehaviorSubject

A Subject that requires an initial value and always replays its most recent value to new subscribers, making it ideal for representing current state.

ReplaySubject

A Subject that records a configurable number of past emissions and replays them to any new subscriber, useful for caching a history of events.

AsyncSubject

A Subject that only emits its final value, and only after the source completes — rarely used, mainly for "give me the last result once done" scenarios.

async pipe

A template pipe that subscribes to an Observable (or Promise) automatically, unwraps its value for display, and unsubscribes when the component is destroyed.

switchMap

Maps to a new inner observable and cancels the previous one if a new value arrives — ideal for typeahead search or route-param-driven HTTP calls.

mergeMap (flatMap)

Maps to inner observables and runs them concurrently without cancelling previous ones, useful when overlapping async operations are all needed.

concatMap

Maps to inner observables and runs them strictly in sequence, waiting for one to complete before starting the next — used when call order matters.

exhaustMap

Maps to an inner observable but ignores new source emissions until the current inner observable completes — useful to prevent duplicate form submissions.

debounceTime / distinctUntilChanged

`debounceTime` waits for a pause in emissions before emitting the latest value; `distinctUntilChanged` filters out consecutive duplicate values — commonly combined for search inputs.

combineLatest

Combines multiple observables and emits an array of their latest values whenever any one of them emits, once all have emitted at least once.

forkJoin

Waits for all given observables to complete and then emits a single array of their final values — the RxJS equivalent of `Promise.all`.

takeUntil

A common unsubscription pattern where a subject emits on component destroy, causing all piped subscriptions to automatically complete and clean up.

Cold vs Hot Observable

Cold observables create a new producer for each subscriber (e.g. HTTP calls); hot observables share one producer across all subscribers (e.g. Subjects, DOM events).

6. Signals (Modern Angular)

Signal

A reactive, synchronous wrapper around a value that notifies dependents when it changes. Read it by calling it as a function, e.g. `count()`.

signal()

Creates a writable signal with an initial value, updated via `.set(value)` to replace it or `.update(fn)` to derive the next value from the current one.

computed()

Creates a read-only signal derived from other signals, automatically recalculating and memoizing its value whenever a dependency changes.

effect()

Registers a function that automatically re-runs whenever any signal it reads changes, typically used for side effects like logging or syncing with non-reactive APIs.

Signal Inputs (input())

A modern alternative to `@Input()` using `input()`, exposing component inputs as read-only signals that integrate directly into the reactive graph.

model()

Defines a signal-based input that also supports two-way binding, replacing the older `@Input` / `@Output` pair pattern for two-way data flow.

Signal vs Observable

Signals are synchronous, pull-based, and glitch-free for UI state; Observables are async-capable, push-based, and better suited for events, HTTP, and complex async pipelines.

toSignal / toObservable

Interop utilities from `@angular/core/rxjs-interop` that convert an Observable into a Signal (and vice versa), bridging the two reactivity models.

7. Forms

Template-Driven Forms

Forms built mostly in the HTML template using directives like `ngModel`, with Angular creating the form model implicitly — best for simple forms.

Reactive Forms

Forms defined explicitly in the component class using `FormGroup` / `FormControl`, giving more predictable, testable, and scalable control over validation and value changes.

FormControl / FormGroup / FormArray

`FormControl` tracks a single field's value and state; `FormGroup` groups controls into an object; `FormArray` manages a dynamic list of controls.

Validators

Functions that check a control's value and return an error object or null, e.g. `Validators.required`. Custom validators follow the same signature for domain-specific rules.

Async Validators

Validators that return a Promise or Observable, used for checks that require a server round-trip, such as verifying a username isn't already taken.

FormBuilder

A service offering shorthand syntax (`fb.group({ ... })`) to construct FormGroups/FormControls more concisely than instantiating them directly.

8. Routing & Lifecycle

Angular Router

Maps URL paths to components, enabling navigation within a single-page application while supporting nested, lazy-loaded, and parameterized routes.

Route Guards

Functions/classes (`CanActivate` , `CanDeactivate` , `CanMatch`) that control whether navigation to or away from a route is allowed, often used for auth checks.

Resolvers

Pre-fetch data before a route activates, so the component receives data already available rather than showing a loading state after navigation.

ActivatedRoute

A service injected into a routed component that exposes route parameters, query params, and data as both snapshot values and observables.

ngOnInit

A lifecycle hook called once after Angular has set the component's initial `@Input` bindings — the standard place for initialization logic.

ngOnChanges

Called whenever an `@Input` property changes, receiving a `SimpleChanges` object with the previous and current values.

ngOnDestroy

Called just before a component is destroyed — the correct place to unsubscribe from observables and clean up timers or event listeners to prevent memory leaks.

ngAfterViewInit / ngAfterContentInit

`ngAfterViewInit` fires after the component's own view (and `@ViewChild` refs) are initialized; `ngAfterContentInit` fires after projected content (`@ContentChild`) is initialized.

9. NgRx & State Management

NgRx Store

A Redux-inspired state management library providing a single, immutable state tree updated only through dispatched actions and pure reducers.

Action

A plain object describing "what happened" (a type and optional payload), dispatched to the store to trigger a state change through reducers and effects.

Reducer

A pure function that takes the current state and an action, and returns a new state object — never mutating the original state directly.

Selector

A memoized function that reads and derives specific slices of state from the store, recalculating only when its underlying state actually changes.

Effect

Handles side effects (like HTTP calls) triggered by dispatched actions, listening on the actions stream and dispatching new actions once the side effect resolves.

NgRx SignalStore

A lightweight, Signal-based state management API that manages local or feature state reactively without the boilerplate of full Store/Effects/Reducers.

Entity Adapter

An NgRx utility that provides standardized CRUD operations and normalized state shape for collections of entities, reducing repetitive reducer code.

10. Testing & Build Tooling

TestBed

Angular's core testing utility that configures a dynamic testing module, allowing components, services, and dependencies to be created and tested in isolation.

Jasmine / Jest

Testing frameworks providing structure (`describe / it`) and assertions (`expect`) for unit tests; Jest is increasingly preferred over Karma/Jasmine for speed.

Playwright / Cypress

End-to-end testing tools that drive a real browser to simulate user interactions and verify full application flows work as expected.

Module Federation

A Webpack feature enabling micro-frontend architectures, letting independently built and deployed Angular apps share code and load each other's modules at runtime.

Nx Monorepo

A build system for managing multiple Angular apps/libraries in one repository, with dependency graph awareness, caching, and affected-only builds/tests.

Ahead-of-Time (AOT) Compilation

Compiles Angular templates to JavaScript during the build step (rather than in-browser), resulting in faster rendering and smaller runtime bundle size.

Prepared as a quick-revision reference for senior Angular interview prep — covers fundamentals through modern Signal-based patterns.